Lattice-Based Encryption: A Case Study on GGH

Jack Hansen, Elen Papyan, and Michael Pham

May 10, 2025

Abstract

The development of quantum computing has led to concerns about the security of many modern-day cryptographic algorithms. As such, significant effort has been made to explore post-quantum cryptography (PQC) in order to protect against these threats. One such example is a lattice-based algorithm: the Goldreich-Goldwasser-Halevi (GGH) Encryption Scheme. In this paper, we explore GGH, its flaws, and recent advancements in lattice-based cryptography post-GGH.

1 Introduction

With the development of technology, communication between multiple parties over large distances becomes increasingly common. A natural question to ask then is *how sensitive information can be sent securely.* In response, asymmetric cryptosystems such as the Diffie-Hellman key exchange [4] and RSA [14] were developed, relying on the hardness of certain problems for their security. More concretely, the Diffie-Hellman key exchange relied on the hardness of the discrete-log problem, while RSA relied on factoring integers being difficult.

However, in 1995, Shor published a paper outlining algorithms which would allow quantum computers to efficiently solve such problems [15]. As research continues and quantum computers improve, a new push has been made to develop post-quantum cryptosystems – algorithms safe from quantum computers.

One such class of post-quantum cryptosystems involves algorithms that rely on the hardness of certain lattice problems for their security. In this paper, we will explore an early attempt at lattice-based cryptography: the Goldreich-Goldwasser-Halevi Cryptosystem. The rest of this paper is structured as follows:

- Section 2 provides the necessary knowledge for understanding the encryption scheme.
- Section 3 describes the GGH Cryptosystem.
- Section 4 details flaws in the scheme, along with attacks performed against it.
- Section 5 analyzes proposed solutions to mitigate the flaws of GGH.
- Section 6 concludes with a survey of more recent attempts at lattice-based cryptography.

2 Prerequisite Knowledge

To begin with, we will provide the necessary knowledge required to understand the inner mechanics of GGH, and hopefully give insight into the following questions:

1. **Why** does the encryption/decryption scheme work?
2. **How** is the scheme secure? **What** metric do we use to base its security off of?

2.1 Lattices

Definition 2.1 (Lattices)**.** Let $v_1, \ldots, v_n \in \mathbb{R}^n$ be a set of linearly independent vectors. Then, the n-dimensional lattice $\mathcal{L}$ generated by $v_1, \ldots, v_n$ is the set of linear combinations of $v_1, \ldots, v_n$ whose coefficients are in $\mathbb{Z}$:

$$\mathcal{L} = \{a_1 v_1 + \ldots + a_n v_n : a_1, \ldots, a_n \in \mathbb{Z}\}$$

We call this set of vectors $v_1, \ldots, v_n$ the *basis* of $\mathcal{L}$. We note that a lattice can have an infinite number of different bases.

Next, we define what a *unimodular matrix* is:

Definition 2.2 (Unimodular Matrix)**.** We say that a matrix U is unimodular if it has interger coefficients and a determinant $\det U = \pm 1$.

This idea of unimodular matrices is important as it allows us to relate two different bases of a lattice $\mathcal{L}$.

Theorem 2.3. *Let B_1, B_2 be two bases. Then, they form the same lattice $\mathcal{L}$ if and only if there exists a unimodular matrix U such that $B_2 = UB_1$.*

Proof. We first prove the forward direction: suppose that B_1, B_2 are two bases for $\mathcal{L}$. Then, by definition of $\mathcal{L}$, there exists square matrices M_1, M_2 with integer coefficients such that:

$$\begin{aligned} B_2 &= M_1 B_1 \\ B_1 &= M_2 B_2 \end{aligned}$$

Combining these equations, we have:

$$\begin{aligned} B_1 &= M_2 M_1 B_1 \\ 0 &= M_2 M_1 B_1 - B_1 \\ 0 &= (M_2 M_1 - I) B_1 \end{aligned}$$

Since B_1 is a basis, each row is linearly independent. So, to have $(M_2M_1 - I)B_1 = 0$, it must be that $M_2M_1 - I = 0 \implies M_2M_1 = I$. Then, we have:

$$\begin{aligned} I &= M_2 M_1 \\ \det I &= \det(M_2 M_1) \\ 1 &= \det(M_2)\det(M_1) \end{aligned}$$

Since M_1, M_2 are matrices with integer entries, their determinants are also integers; in other words, $\det(M_1) = \det(M_2) = \pm 1$. Thus, M_1, M_2 are unimodular matrices.

For the backwards direction, we first introduce the following lemma:

Lemma 2.4. *If U is unimodular, then U^{-1} is as well.*

Proof. First, we want to show that U^{-1} has integer coefficients. For this, we recall that $U^{-1} = \frac{1}{\det U} C^{\intercal}$, where C is the cofactor matrix.

Since $U \in \mathbb{Z}^{n \times n}$, the determinant of each sub-matrix of U must also be an integer. Then, by construction, each entry in our cofactor matrix is also an integer. And since $\det U = \pm 1$, this means that U^{-1} will have integer coefficients only.

To show that $\det U^{-1} = \pm 1$, observe that $\det U^{-1} = \frac{1}{\det U} = \frac{1}{\pm 1} = \pm 1$. Thus, U^{-1} is unimodular. □

Using this lemma, we now prove the backward direction. Suppose we have a unimodular matrix U such that $B_2 = UB_1$. Then, because U has integer coefficients, this implies then that each row in B_2 is an integer linear combination of rows in B_1. In other words, $\mathcal{L}(B_2) \subseteq \mathcal{L}(B_1)$, where $\mathcal{L}(B)$ is the lattice generated by the basis B.

On the other hand, $B_1 = U^{-1}B_2$. Since U^{-1} is unimodular, a similar argument can be used to show that $\mathcal{L}(B_1) \subseteq \mathcal{L}(B_2)$. Thus, they form the same lattice as desired. ■

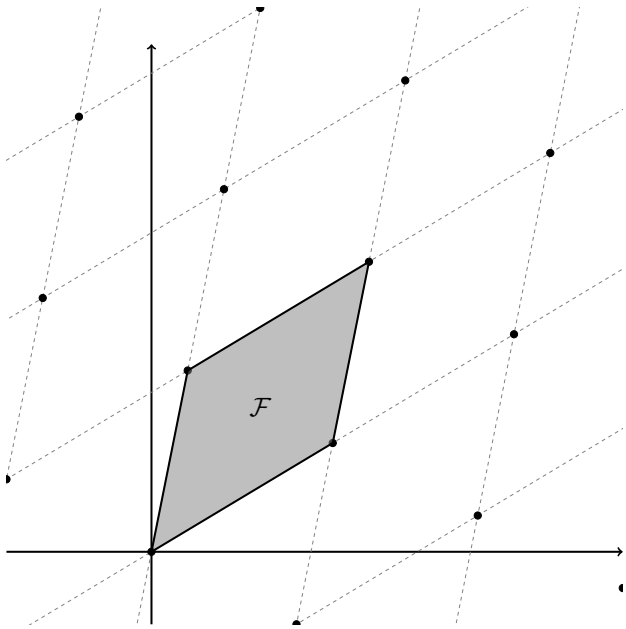

Figure 1: *Lattice $\mathcal{L}$ and its fundamental domain $\mathcal{F}$.*

So far, we've been looking at lattices in terms of vectors. However, we can view them under a group-theoretic lens as well:

Definition 2.5 (Lattice as Groups)**.** An n-dimensional lattice $\mathcal{L}$ is a subset of $\mathbb{R}^n$ that has the following properties:

- Additive Subgroup
- Discrete: There exists an $\varepsilon > 0$ such that for all $x \in \mathcal{L}$, there are no other lattice points in the ε-neighborhood around it.

Viewing lattices as groups gives us a more concrete way to visualize them, as seen in Figure 1.

A key property of lattices is its *fundamental domain.*

Definition 2.6 (Fundamental Domain)**.** Suppose we have an n-dimensional lattice $\mathcal{L}$ with basis $v_1, \ldots, v_n$. Then, the *fundamental domain* is defined as the set

$$\mathcal{F}(v_1, \ldots, v_n) = \{\lambda_1 v_1 + \cdots + \lambda_n v_n : 0 \le \lambda_i < 1\}$$

In Figure 1, we see that the fundamental domain $\mathcal{F}$ is the shaded region of our lattice. The fundamental domain is of great interest to us, in part due to the following theorem:

Theorem 2.7. *Let $\mathcal{L} \subset \mathbb{R}^n$ be an n-dimensional lattice. Denote its fundamental domain by $\mathcal{F}$. Then, every vector $w \in \mathbb{R}^n$ can be expressed as*

$$w = u + v \quad \textit{for a unique } u \in \mathcal{F} \textit{ and unique } v \in \mathcal{L}.$$

Proof. First, we will show that any arbitrary $w \in \mathbb{R}^n$ is the sum of some vector $u \in \mathcal{F}$, and some lattice point $v \in \mathcal{L}$.

Denote $v_1, \ldots, v_n$ to be the basis for our lattice $\mathcal{L}$. Then, by definition, this set of vectors must be linearly

independent. Because there are n linearly independent vectors, this must in fact be a spanning set for $\mathbb{R}^n$ as well.

In other words, $v_1, \ldots, v_n$ is also a basis for $\mathbb{R}^n$. With that in mind, we can express w as a linear combination of $v_1, \ldots, v_n$:

$$w = \alpha_1 v_1 + \cdots + \alpha_n v_n, \quad \alpha_i \in \mathbb{R}$$

Since each scalar α_i is real, we can "extract" the integer and non-integer part of it. That is, we can express $\alpha_i = \lambda_i + z_i$, where $0 \leq \lambda_i < 1$, and $z_i \in \mathbb{Z}$.

With that in mind, we now have:

$$\begin{aligned} w &= (\lambda_1 + z_1)v_1 + \cdots + (\lambda_n + z_n)v_n \\ &= \lambda_1 v_1 + z_1 v_1 + \cdots + \lambda_n v_n + z_n v_n \\ &= (\lambda_1 v_1 + \cdots + \lambda_n v_n) + (z_1 v_1 + \cdots + z_n v_n) \end{aligned}$$

Notice then that $u = \lambda_1 v_1 + \cdots + \lambda_n v_n$ is a vector that is a part of our fundamental domain $\mathcal{F}$. Meanwhile, since each z_i is an integer, $v = z_1 v_1 + \cdots + z_n v_n$ is precisely a lattice point in $\mathcal{L}$. Thus, $w = u + v$ for $u \in \mathcal{F}$ and $v \in \mathcal{L}$ as desired.

For uniqueness, suppose we have $w = u + v = u' + v'$. Then, this means that we have:

$$\alpha_1 v_1 + \cdots + \alpha_n v_n = \alpha_1' v_1 + \cdots + \alpha_n' v_n$$

Now, since $v_1, \ldots, v_n$ are linearly independent, this means that $\alpha_i = \alpha_i'$ for all $i \in \{1, \ldots, n\}$. In other words, we must have:

$$\begin{aligned} \alpha_i &= \alpha_i' \\ \lambda_i + z_i &= \lambda_i' + z_i' \\ \lambda_i - \lambda_i' &= z_i' - z_i \end{aligned}$$

Since $z_i, z_i' \in \mathbb{Z}$, we have that $z_i' - z_i \in \mathbb{Z}$ too. Recall now that $0 \leq \lambda_i, \lambda_i' < 1$; for $\lambda_i - \lambda_i' \in \mathbb{Z}$, it must be that $\lambda_i = \lambda_i'$. So, we have $u = u'$. Furthermore, this must mean then that:

$$v = w - u = w - u' = v'$$

Thus, we have uniquenesss as well. ■

Intuitively, this result tells us that if we translate our fundamental domain around, we can cover $\mathbb{R}^n$. This can be seen in Figure 1, where the translated fundamental domain is depicted with the dotted outlines.

With this notion of the fundamental domain, we can now define the determinant of our lattice $\mathcal{L}$:

Definition 2.8 (Determinant of Lattice)**.** Let $\mathcal{L}$ be an n-dimensional lattice with fundamental domain $\mathcal{F}$. Then, the n-dimensional volume of $\mathcal{F}$ is the determinant of our lattice, $\det \mathcal{L}$.

From this definition, if we imagine $v_1, \ldots, v_n$ to be the sides of this parallepiped, it's clear that the largest volume that can be obtained by them is when they are pairwise orthogonal. This leads us to the following upper-bound for $\det \mathcal{L}$:

Proposition 2.9 (Hadamard's Inequality)**.** *Let $\mathcal{L}$ be a lattice, and $v_1, \ldots, v_n$ be a basis for it. Let $\mathcal{F}$ be the fundamental domain. Then, we have the following inequality:*

$$\det \mathcal{L} = \operatorname{vol} \mathcal{F} \leq \|v_1\| \cdots \|v_n\| .$$

We observe that the closer our basis vectors are to being pairwise orthogonal, the closer Hadamard's Inequality is to becoming an equality.

Now, as it turns out, the determinant of our lattice is invariant; that is, it doesn't depend on the basis we choose for $\mathcal{L}$ [10].

With all of this in mind, we are now able to quantify the "orthogonality" of a lattice's basis:

Definition 2.10 (Hadamard Ratio)**.** We define the Hadamard Ratio of the basis $B = \{v_1, \ldots, v_n\}$ to be:

$$\mathcal{H}(B) = \left(\frac{\det \mathcal{L}}{\|v_1\| \cdots \|v_n\|} \right)^{1/n}$$

Thus, we see that $0 < \mathcal{H}(B) \leq 1$. The closer this ratio is to being 1, the more orthogonal our basis is. This result is of particular importance, as we will see in Section 2.4.1.

2.2 Complexity Theory

Before we discuss the lattice problem that GGH relies on, it is important to discuss the notion of "hardness" that we have been alluding to.

We have said that cryptosystems rely on the "hardness" of certain problems for its security. However, what does this actually mean? What makes a problem "hard"...?

This is where Complexity Theory comes into play, as it tries to quantify the difficulty of a problem. Loosely speaking, a problem is *efficiently solvable* if it is solvable in polynomial time $\mathcal{O}(n^k)$. Then, this allows us to define the following two *complexity classes*:

- P: This is the set of problems which are efficiently solvable.
- NP: This is the set of problems whose solutions are efficiently verifiable.

Remark. Typically, the classes of P and NP are used to describe *Decision Problems* (i.e. problems whose solution is "yes" or "no"). However, for this paper, we will use them to refer to general computational problems.

Notice that all problems that can be solved efficiently can also be verified efficiently. This tells us that P $\subseteq$ NP. For this paper, when we say that a problem is in NP, we mean that it is in NP but not P. On the other hand, we don't know if NP $\subseteq$ P; this is the open-question of P $\stackrel{?}{=}$ NP in Theoretical Computer Science!

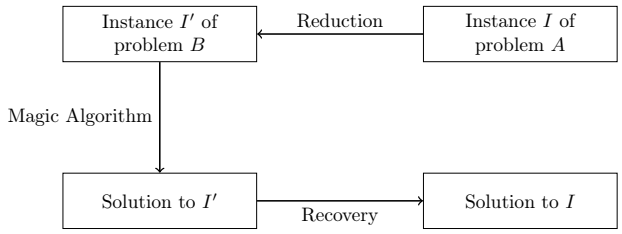

Figure 2: *Reduction Diagram.*

This notion of complexity classes gives us an idea of how we can quantify the difficulty of a problem: if it is in P, it's easy. Otherwise, it is hard. Now, since the problem of P versus NP is still unsolved, it's useful to look at problems in terms of their relative difficulty instead. This is where the idea of *reductions* come into play.

Definition 2.11 (Reductions)**.** We say that a problem A can be reduced to B if there exists a **polynomial-time** algorithm that transforms an instance of A to an instance of B.

Basically, if we have some magic algorithm that allows us to solve B, but we are interested in A instead, if we can reduce our instance of A to an instance of B, we can then apply the algorithm and solve for A as well.

Notice that if we can reduce A to B, this tells us that the difficulty of A is **at most** B. That is, $A \leq B$. Intuitively, this is because we are using a black-box algorithm that solves B to solve A; this means A's runtime ultimately is at worst equal to the magic algorithm's runtime.

With this idea of reductions in mind, we can introduce the last two relevant complexity classes for this paper:

- NP-Hard: This is the class of problems in which all problems in NP reduce to.
- NP-Complete: This includes all problems that are both NP and NP-Hard.

Going back to Cryptography, we see that cryptosystems use Complexity Theory as a way to quantify how secure their schemes are, basing the security of their schemes on if the encrypted instance is efficiently solvable.

Example 2.12 (RSA)**.** To better illustrate how these two areas are connected, we turn to the famous problem of factoring. The factoring problem involves finding a prime factorization for an $n \in \mathbb{Z}$ (if it exists). Currently, there is no known polynomial-time algorithm to solve it, making it a part of NP [13].

With this problem as its basis, in 1978, Rivest et al. constructed the RSA cryptosystem [14]. Here, the security relies on the fact that an attacker cannot reasonably guess the primes p, q that creates the public key n – this assumption stems directly from the presumed hardness of the factoring problem!

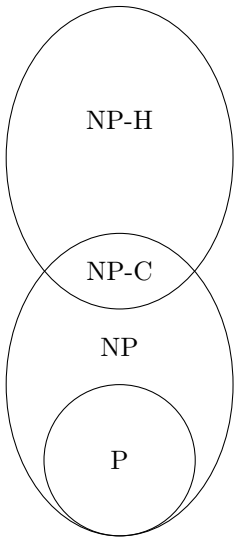

Figure 3: *Complexity Classes assuming* P $\neq$ NP.

This example illustrates both the connection between Cryptography and Complexity Theory, but also how cryptosystems are only as secure as the underlying problem it is built off of. If factoring turns out to be in P, RSA is effectively broken.

With that in mind, it would be desireable to be able to build schemes using NP-Hard problems. However, in practice, it is hard to do so; cryptosystems not only require a strong encryption scheme, but also an easy decryption scheme. As a result, many base their protocols off of specific cases of NP-Hard problems – and these instances may not necessarily be NP-Hard.

Remark. A massive caveat to this section is that Complexity Theory is interested in the "worst-case" scenario. That is, we say that a problem is NP if there doesn't exist an algorithm that can solve *all* instances of it in polynomial time. This means that it's possible for a problem to be in NP despite most of its instances being easy-to-solve. This idea is called *worst-case hardness.*

In contrast, cryptosystems aim for protocols that are *hard on average.* That is, given a random instance, it is expected to be hard to solve. Unlike Complexity Theory, Cryptography is interested in *average-case hardness.*

2.3 Lattice-Based Problems

In this subsection, we will describe two closely related lattice problems: the *Closest Vector Problem* and the *Shortest Vector Problem.*

2.3.1 The Closest Vector Problem

Now, we introduce the core of GGH's security: the Closest Vector Problem (CVP).

Definition 2.13 (Closest Vector Problem)**.** Suppose we have a lattice $\mathcal{L}$ and vector space V. Then, given

some arbitrary vector $v \in V$, we want to find the point $l \in \mathcal{L}$ closest to v. In other words, we want to find $l \in \mathcal{L}$ that minimizes the Euclidean norm $\|v - l\|$.

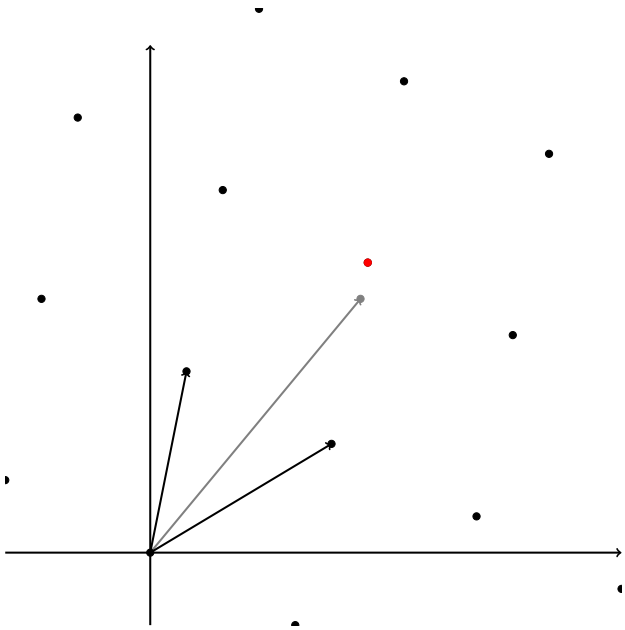

Figure 4: *Instance of the Closest Vector Problem.*

As it turns out, CVP has desirable properties for being the basis of a cryptosystem.

- Firstly, CVP is an NP-Hard problem. This was first proven in 1981 by Emde Boas [5].
- Secondly, in 1997, Ajtai and Dwork constructed a cryptosystem based off of CVP with an average-case to worst-case equivalence [1].
  - They showed that breaking a random instance of the problem was equivalent to solving the worst-case.

Nevertheless, despite CVP being NP-Hard, in practice cryptosystems typically rely on easier subclasses of the problem in order for there to be a reasonable decryption algorithm.

Furthermore, CVP being NP-Hard doesn't mean that all instances of it are hard to solve. In fact, in Section 2.4, we will explore algorithms that efficiently solves easier CVP instances with high probability. As such, it is important when designing a cryptosystem to ensure that it **can't be reduced to a simplified instance that's solvable efficiently**.

2.3.2 The Shortest Vector Problem

Another lattice-based problem closely linked to CVP is the Shortest Vector Problem (SVP).

Definition 2.14 (Shortest Vector Problem)**.** Given a lattice $\mathcal{L}$, we want a nonzero vector $v \in \mathcal{L}$ which minimizes $\|v\|$.

There are some interesting properties of SVP, such as it being at most as hard as CVP [7]. However, the main reason we are interested in SVP here will be seen in Section 4.2, where we use the solution to SVP to solve for CVP.

2.4 Solving (Easy) CVP

As we've hinted at in the previous subsection, not all instances of CVP are hard to solve. Here, we discuss some ways to efficiently solve certain CVP instances.

2.4.1 Babai's Rounding Technique

Let's consider a lattice $\mathcal{L}$ with a basis $v_1, \ldots, v_n$ such that the vectors are pairwise orthogonal; meaning that $v_i \cdot v_j = 0$ for all $i \neq j$. As we will see the naïve idea of rounding to solve CVP will be fruitful in these conditions.

Suppose we ask for the closest vector to $w \in \mathbb{R}^n$ with respect to the orthogonal basis representing our lattice. For an arbitrary $v \in \mathcal{L}$, what does $\|w - v\|$ look like in this case? First, let's write

$$\begin{aligned} w &= \alpha_1 v_1 + \cdots + \alpha_n v_n, \quad \alpha_i \in \mathbb{R} \\ v &= \beta_1 v_1 + \cdots + \beta_n v_n, \quad \beta_i \in \mathbb{Z} \end{aligned}$$

Then we can write the norm as

$$\begin{aligned} \|w - v\| &= \|(\alpha_1 - \beta_1)v_1 + (\alpha_2 - \beta_2)v_2 \\ &\quad + \cdots + (\alpha_n - \beta_n)v_n\| \\ &= \sum_{i=1}^{n}\sum_{j=1}^{n}(\alpha_i - \beta_i)(\alpha_j - \beta_j)v_i v_j \\ &= \sum_{i=1}^{n}(\alpha_i - \beta_i)^2 v_i^2 \qquad (1) \\ &= \sum_{i=1}^{n}(\alpha_i - \beta_i)^2 \|v_i\|^2 \end{aligned}$$

where (1) follows from every $i \neq j$ term being 0. This is thanks to our choice of orthogonal basis! In order to find the coefficients which make v the closest lattice point to w, we focus on minimizing $(\alpha_i - \beta_j)$. This requires choosing our β_i to be the integer closest to α_i so we round α_i and that becomes β_i.

This procedure works because of the underlying structure that comes with a lattice, in particular the fundamental domain and its properties.

We know from Theorem 2.7 that any $w \in \mathbb{R}^n$ can be uniquely written as the sum of a vector in the fundamental domain and a vector in $\mathcal{L}$. This can also be thought of as w being inside of the translation of the fundamental domain by the unique vector in $\mathcal{L}$ [10]. Now when the basis for $\mathcal{L}$ is orthogonal (or close to), we can look at the vertices of the translated fundamental domain and the closest lattice point will be one of

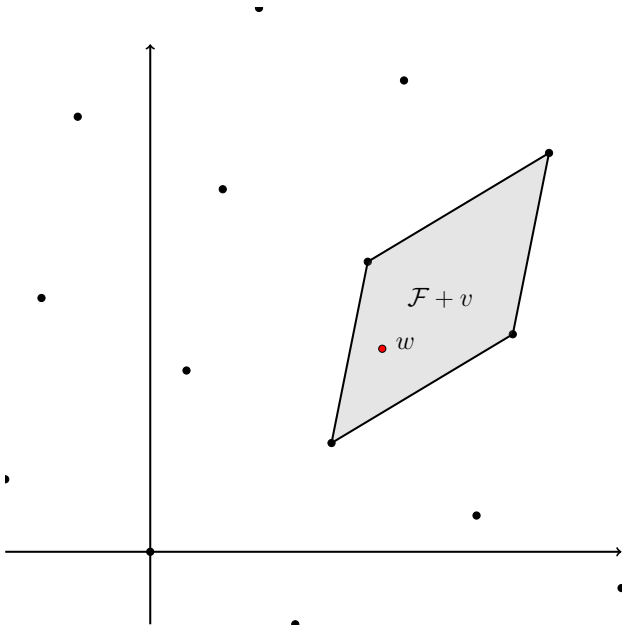

Figure 5: *Solving CVP of w with the fundamental domain $\mathcal{F}$ shifted by $v \in \mathcal{L}$.*

these vertices, which we get by simply rounding the coefficients of w. This is illustrated in Figure 5.

This leads us to Babai's Algorithm, which allows us to solve these simplified instances of CVP.

Algorithm 1 Babai's Closest Vector Algorithm

Input: $\{v_1, \ldots, v_n\}$ basis for $\mathcal{L}$, $w \in \mathbb{R}^n$
Output: closest $l \in \mathcal{L}$ to w

Write $w = t_1 v_n + \cdots + t_n v_n$ with $t_i \in \mathbb{R}$
for $i = 0$ to n **do**
    Round t_i to the nearest integer
    Save that value in variable α_i
end for
return $l = \alpha_1 v_1 + \cdots + \alpha_n v_n$

It's very important to emphasize that this technique does **not** work reliably if the basis vectors are not sufficiently orthogonal. Furthermore, as the dimension n increases, the worse this algorithm becomes at solving CVP.

2.4.2 LLL Algorithm

As we saw in the section above, Babai's allows us to approximate CVP given that we provide the algorithm with a good basis. What if we only have access to a bad basis? Is there a way to turn a highly nonorthogonal basis to something closer to orthogonal? Introducing, lattice reduction algorithms! In particular, we'll be discussing the Lenstra-Lenstra-Lovász (LLL) lattice reduction algorithm.

First we introduce the algorithm, then dive into its intricacies.

Algorithm 2 LLL Algorithm [10]

Input: $\{v_1, \ldots, v_n\}$ basis for $\mathcal{L}$
Output: $\{w_1, \ldots, w_n\}$ LLL reduced basis for $\mathcal{L}$

1: Set $k = 2$
2: Run Gram-Schmidt on $v_1, \ldots v_n$ to get $v_1^*, \ldots, v_n^*$
3: **while** $k \leq n$ **do**
4: **for** $j = k-1, k-2, \ldots, 2, 1$ **do**
5: Set $v_k = v_k - \lfloor \mu_{k,j} \rceil v_j$
6: **end for**
7: **if** $\|v_k^*\|^2 \geq (\frac{3}{4} - \mu_{k,k-1}^2) \left\|v_{k-1}^*\right\|^2$ **then**
8: $k = k + 1$
9: **else**
10: Swap v_{k-1} and v_k
11: Set $k = \max(k-1, 2)$
12: **end if**
13: **end while**
14: **return** the modified $v_1, \ldots, v_n$

Whenever any v_i changes, we rerun Gram-Schmidt to get the updated v_i^* values.
Note that $\mu_{i,j} = (v_i \cdot v_j^*) / \left\|v_j^*\right\|^2$

We cannot directly use Gram-Schmidt to construct a good basis since Gram-Schmidt uses nonintegral values in its process to give us a basis for the *vector space* that is spanned by the vectors given. However, we use the result of Gram-Schmidt to define what it means to be LLL *reduced*.

Definition 2.15. A basis $\mathcal{B}$ is LLL *reduced* if for all $1 \leq i, j \leq n$ it satisfies the **size** (2) and **Lovász** (3) conditions:

$$|\mu_{i,j}| = \frac{|v_i \cdot v_j^*|}{\left\|v_j^*\right\|^2} \leq \frac{1}{2} \tag{2}$$

$$\|v_i^*\|^2 \geq (\frac{3}{4} - \mu_{i,i-1}^2)) \left\|v_{i-1}^*\right\| \tag{3}$$

In the algorithm, lines 4-6 represent the size condition and lines 7-8 represent the Lovász condition. The swap at lines 10-11 is for further size reductions in order for the Lovász condition to be satisfied.

The key result that Lenstra, Lenstra, and Lovász proved in their paper was that this algorithm does in fact produce a good basis and is computable in polynomial time [10].

Remark. If we replace the constant fraction $\frac{3}{4}$ by any value that is **strictly** less than 1, then LLL still runs in polynomial time. In practice, a value larger than $\frac{3}{4}$ but less than 1 is used. If we did replace the constant value with 1, then we hit an open problem as to whether or not LLL terminates in polynomial time.

In practice, this algorithm has challenges in being efficiently implemented. One of these challenges is that with higher dimensions of the lattice you are working with, intermediate values become huge; thus requiring floating point approximations. However, this leads to issues regarding round-off errors [10].

3 The GGH Cryptosystem

With the relevant prerequisite knowledge in mind, we will now delve into the GGH Cryptosystem. In this section, we will discuss the set-up for the scheme, along with the actual encryption and decryption process.

3.1 Set-Up

Because GGH is a public-key encryption system, we must have a public and private key pair. The scheme relies on CVP to ensure that the encrypted message isn't solvable unless one has a good basis, as discussed in Section 2.4.1.

To begin with, our private keys are as follows:

- First, we have a "good" basis B. This is defined to be a basis that is short and nearly orthogonal.
  - We can use Definition 2.10 to measure how orthogonal our basis is. The closer the basis' Hadamard Ratio is to 1, the more orthogonal it is.
- Additionally, we also require a randomly-generated, unimodular matrix U.
  - This can be generated by taking a random combination of elementary matrices [10].

For the public key, we will have a "bad" basis B' defined as follows:

$$B' = UB$$

Recall Theorem 2.3, which tells us that bases for a lattice $\mathcal{L}$ are related through a unimodular matrix U. Crucially, this means that B' preserves the lattice structure, acting as a new basis for the lattice. Essentially, U is a change of basis matrix.

When we hit B with U, the resulting basis will be stretched out and highly non-orthogonal, making it a "bad" basis.

Example 3.1 (Setup)**.** We begin by demonstrating how we can generate our public and private keys for GGH in a 2-dimensional lattice. In this scenario, suppose we have two parties: Alice and Bob. Bob wants to send a message to Alice using GGH.

First, we define Alice's private basis to be:

$$v_1 = (17, 0), \quad v_2 = (0, 19)$$

We note that since these two vectors are orthogonal to each other, its Hadamard Ratio is precisely equal to 1. Now, let our unimodular matrix U be:

$$U = \begin{pmatrix} 2 & 3 \\ 3 & 5 \end{pmatrix}$$

Then, we can calculate the public basis B' that Bob will use to encrypt his messages sent to Alice:

$$\begin{aligned} B' &= UB \\ &= \begin{pmatrix} 34 & 57 \\ 51 & 95 \end{pmatrix} \end{aligned}$$

Notice that the Hadamard Ratio for B' is:

$$\begin{aligned} \mathcal{H}(B') &\approx \left(\frac{323}{7156}\right)^{1/2} \\ &\approx 0.212 \end{aligned}$$

As desired, the bad basis B' has a low Hadamard Ratio, and isn't nearly orthogonal. We also note that, clearly, the lengths of w_1, w_2 are much longer than v_1, v_2. Thus, we see how this unimodular matrix U can transform our "good" basis into a "bad" one.

3.2 Encryption

Suppose Bob wants to send Alice a message m represented by $(m_1, \ldots, m_n)$ where $m_i \in \mathbb{Z}$. In order to encrypt his message, Bob also needs to randomly generate an error vector $e \in \{-\sigma, \sigma\}^n$. Note that σ is a predetermined and **public** value (in the paper for GGH, σ was set to 3). Now with all the information in place, Bob calculates

$$c = mB' + e$$

This resulting ciphertext is simply a point on the lattice that has been shifted by some small amount. Because the attacker only knows of the bad basis B', they aren't able to use techniques such as Babai's Algorithm to eliminate the error vector. Thus, in theory, the message is safe from those who don't know the "good" basis. This is reminiscent of CVP, and the authors of GGH conjectured that for $n > 300$, the scheme could not be solved efficiently [8].

Remark. Although the noise parameter σ is known, the actual error vector isn't; each of its entries is sampled uniformly at random from $\{-\sigma, \sigma\}$.

Example 3.2 (Encryption)**.** Continuing from Example 3.1, Bob now wants to send a message to Alice. For this scheme, suppose that we are using $\sigma = 1$, and for this particular instance, we have generated an error vector $e = (1, -1)$.

Now, suppose Bob wants to send the message $m = (2, -5)$ to Alice. Then, he can encrypt his message using GGH as follows:

$$\begin{aligned} c &= mB' + e \\ &= (2, -5)\begin{pmatrix} 34 & 57 \\ 51 & 95 \end{pmatrix} + (1, -1) \\ &= (-186, -362) \end{aligned}$$

Thus, Bob will send the ciphertext $c = (-186, -362)$ to Alice.

3.3 Decryption

To decrypt, we essentially have to solve CVP with the real valued vector being the ciphertext, but this is not difficult for Alice! To decrypt a message c, Alice can choose two routes: apply Babai's rounding technique directly to the ciphertext, or to a modified ciphertext.

Suppose Alice apply Babai's rounding technique at the start using her "good" basis and c as the inputs, then

$$c = mB' + e \to c = mB'$$

Since Alice's basis is (nearly) orthogonal, the error term will vanish, leaving us with the point on the lattice that the message is sent to. To isolate the message, Alice finds B^{-1} and U^{-1}, then

$$cB^{-1} = mU$$
$$cB^{-1}U^{-1} = m$$

She recovered Bob's message!

Suppose that Eve gains access to the ciphertext and wants to decrypt Bob's message. She could also attempt to apply Babai's rounding technique, however Eve only knows the public (non-orthogonal) basis, B', this ideally will not lead to the correct CVP output.

Example 3.3 (Decryption)**.** Carrying on from Example 3.2, suppose Alice has now received Bob's (encrypted) message. She now wants to decrypt it and read its content.

Because Alice's private basis is orthogonal, she can first apply Babai's Rounding Technique using this basis to remove the error term. More concretely, she first expresses the ciphertext as a linear combination of the private basis B using real coefficients:

$$c \approx -10.94v_1 + -19.05v_2$$

Then, using Babai's, she can round these coefficients to the nearest integer, then calculate the closest lattice vector

$$v = -11v_1 + -19v_2 = (-187, -361)$$

Now, she can simply calculate B'^{-1} and recover the original message m:

$$\begin{aligned} vB'^{-1} &\approx (-187, -361)\begin{pmatrix} 0.294 & -0.176 \\ -0.158 & 0.105 \end{pmatrix} \\ &= (2, -5) \end{aligned}$$

Thus, Alice successfully decrypted Bob's message!

4 Vulnerabilities

Throughout the course of our description of the scheme in Section 3, we've hinted at the fact that the original authors had set parameters for the dimension n, along with the σ used for their error vectors.

In this section, we will illustrate why the choice of parameters is important, and show why the scheme ultimately failed due to a flaw in its construction.

4.1 Low Dimensionality

When implementing GGH, it is important to choose a sufficiently large dimension. Because of the LLL Algorithm, attackers are able to reduce a "bad" basis into a significantly better one. In fact, in [10], they provide an example where the LLL-reduced basis is better than the public basis. As such, the authors of GGH recommended using a dimension of $n \geq 300$ [8].

4.2 Short Error Vectors

Now, we illustrate the dangers of using a short error vector by revisiting the scenario in Example 3.2.

Example 4.1 (Eavesdropper)**.** Suppose we have a third party, Eve, who intercepted Bob's message. Eve wants to know what Bob is trying to tell Alice.

Ideally, Eve shouldn't be able to crack the message. However, this example will illustrate why it is important for the scheme to use sufficiently large error vectors.

Suppose that Eve tried to use Babai's Algorithm using the known bad basis B' instead. This would yield us:

$$c \approx 2.45w_1 + -5.28w_2$$

Rounding with Babai's, Eve now gets:

$$w = 2w_1 + -5w_2 = (-187, -361)$$

However, as seen from Example 3.3, $w = v$; Eve can in fact recover Bob's original message!

This illustrates the need to choose sufficiently large error vectors, or else attackers can simply apply Babai's and solve the CVP instance.

Short vectors also give rise to the embedding attack described in the original GGH paper [8], which embeds a lattice into one higher dimension, plus a specified vector c. More precisely, if $(b_1 \dots b_n) \in \mathbb{Z}^n$ forms a basis for L and we're trying to find the closest point in L to c, we form the lattice L' and the corresponding matrix

$$B' = \begin{bmatrix} b_{11} & \cdots & b_{n1} & c_1 \\ \vdots & \ddots & \vdots & \vdots \\ b_{n1} & \cdots & b_{nn} & c_n \\ 0 & \cdots & 0 & 1 \end{bmatrix}$$

We've now reduced our instance of CVP in n dimensions to an instance of SVP in $n+1$ dimensions. If $v \in L$ is the vector which minimizes the distance to c, then $(v - c, 1) \in L'$. In the case where c is close to v, then $(v - c, 1)$ is likely to be the shortest vector in L', allowing us to find v efficiently. We can find this using algorithms such as LLL.

As such, the longer the error vector is, the more resilient it is against embedding attacks. However, the risk of decryption errors becomes larger.

4.3 Nguyen's Attack

In their original paper, Goldreich et al. conjectured that their scheme was secure for parameters $(n, \sigma) = (300, 3)$. In order to prove the security of their cryptosystem, they published a list of "Internet Challenges." If these challenges proved to be unbreakable, their scheme would be secure.

However, just two years after the GGH Scheme was published, Nguyen discovered an attack which leveraged the predictability of the scheme's error vector [12]. The crux of this attack lies in the following observation:

Observation. Since $c = mB' + e$, where each entry of e is sampled uniformly at random from the set $\{-\sigma, \sigma\}$, we can leverage this predictability to erase the noise vector.

Because each entry of our noise vector is $\pm\sigma$, by adding in a vector $s = (\sigma_1, \ldots, \sigma_n)$, we will have a new error vector e' with entries in $\{0, 2\sigma\}$. Therefore, when modded by 2σ, e' will get erased!

Now, with this observation in mind, the path of attack is clear:

1. First, let us define $s = (\sigma_1, \ldots, \sigma_n)$.
2. Next, we compute $c' = c + s$. This yields us:

$$\begin{aligned} c' &= c + s \\ &= (mB' + e) + s \\ &\equiv mB' \pmod{2\sigma} \end{aligned}$$

3. From here, we find $m' = m \pmod{2\sigma}$.
   - If $B' \pmod{2\sigma}$ is invertible, we can simply compute the inverse and calculate $mB'B'^{-1} \pmod{2\sigma} = m \pmod{2\sigma}$. As it turns out, for $\sigma = 3$, the probability that $B' \pmod{2\sigma}$ is invertible is around 20% [12].
   - Otherwise, we instead have a system of linear equations $c' = mB' \pmod{2\sigma}$. Here, we want to find unknown m with known c', B'. Methods exist to solve this, as seen in [3] and [9].
4. Now, we can calculate $c - m'B'$:

$$\begin{aligned} c - m'B' &= (mB' + e) - m'B' \\ &= (mB' - m'B') + e \\ &= (m - m')B' + e \end{aligned}$$

5. Recall that m' is the remainder of m modulo 2σ. This means that $m - m' = 2\sigma z$ for some $z \in \mathbb{Z}^n$, Meanwhile, each entry of e is $\pm\sigma$. So, we can divide everything by 2σ to get:

$$\frac{(m - m')B'}{2\sigma} + \frac{e}{2\sigma}$$

6. Because $\frac{e}{2\sigma}$ has entries in $\left\{\pm\frac{1}{2}\right\}$, we have reduced our CVP instance into a simplified instance of CVP. Applying Babai's Algorithm, we can erase this shorter noise vector.
7. Now, we're left with $\frac{c - m'B'}{2\sigma} = \frac{(m - m')B'}{2\sigma}$. Thus, we can easily compute $m = cB'^{-1}$.

5 Mitigations

There are two major issues with the construction of GGH. First, there's a predictable structure with the error vectors, which led to our modulo attack. With that in mind, we might consider adjusting our errors to instead take on values in $[-\sigma, \sigma]$ instead of $\{\pm\sigma\}$. However, this presents its own problem. Picking rational numbers complicates the computations, leading to slower encryption and decryption. Picking integers in $[-\sigma, \sigma]$ alleviates this issue, but cuts the expected magnitude of the error vector almost in half; it becomes more susceptible to embedding attacks.

This leads us to the next problem, which is fundamental to GGH: the error vectors are just too short. Nguyen solved instances of CVP induced by GGH in dimensions up to 350 and said "it would be dangerous to use a dimension less than 400" [12]. This problem cannot be fixed, as the $\sigma = 3$ used in most implementations maximizes the magnitude of the error vector while avoiding almost all decryption errors.

So, if we can't add more randomness to the error or lengthen it, can we increase the dimension? Unfortunately, dimension 400 already has a public key size of 1.8Mb, over 3500 times the size of a standard 4096-bit RSA public key, making any increase to key-size impractical without changes to the system.

There have been attempts to tweak the GGH system to increase security. We will briefly discuss two, one of which finds a way to increase the length of the error vector and the other which makes higher dimensions more feasible.

Masayuki Yoshino and Noboru Kunihiro proposed an alternative scheme which allowed for larger error vectors [17]. In the typical GGH scheme, $\lceil B^{-1}e \rfloor$ is the zero vector, but changes to decryption allow for $\lceil B^{-1}e \rfloor$ to take on some values of ± 1.

Another approach uses objects called Generalized Low Density Lattices, which allow them to drastically cut key sizes [11]. With this technique, the authors were able to reduce the key size for $n = 1000$ from 43.6 MB to around 100 KB.

6 Conclusion

Throughout this paper, we've shown that the GGH Cryptosystem is fundamentally flawed due to its predictable error vectors and large key sizes. Goldreich et al. argued the cost of large key sizes were outweighed by the speed gains, but any sufficiently large dimension

has an impractical key size. In private communication between Nguyen and Goldreich, Goldreich declared the system "dead" [12].

With that in mind, we must ask the question: *is lattice cryptography itself dead?* The answer is a resounding no, lattice cryptography is alive and well. In fact, it's one of the most promising avenues for post-quantum cryptography. Some of the more modern uses of lattices include Kyber [2], Gentry's Fully Homomorphic Encryption scheme [6], and Falcon [16]. Kyber is a cryptosystem based on Module Learning with Errors (M-LWE), Gentry's is based on Ideal Lattices, and Falcon is based on NTRU, another lattice-based cryptosystem invented around the same time as GGH.

References

[1] Miklós Ajtai and Cynthia Dwork. "A public-key cryptosystem with worst-case/average-case equivalence". In: *Proceedings of the twenty-ninth annual ACM symposium on Theory of computing - STOC '97*. STOC '97. ACM Press, 1997, pp. 284–293. DOI: 10.1145/258533.258604.

[2] Joppe Bos et al. "CRYSTALS - Kyber: A CCA-Secure Module-Lattice-Based KEM". In: *2018 IEEE European Symposium on Security and Privacy (EuroSamp;P)*. IEEE, Apr. 2018, pp. 353–367. DOI: 10.1109/eurosp.2018.00032.

[3] Henri Cohen. *A Course in Computational Algebraic Number Theory*. 4. print. Graduate texts in mathematics 138. Literaturverz. S. [527] - 539. Berlin: Springer, 2000. 551 pp. ISBN: 9783540556404.

[4] W. Diffie and M. Hellman. "New directions in cryptography". In: *IEEE Transactions on Information Theory* 22.6 (Nov. 1976), pp. 644–654. ISSN: 1557-9654. DOI: 10.1109/tit.1976.1055638.

[5] P. van Emde Boas. *Another NP-complete Partition Problem and the Complexity of Computing Short Vectors in a Lattice*. Mathematical preprints series. Universiteit van Amsterdam. Mathematisch Instituut, 1981. URL: https://books.google.com/books?id=rRcgrgEACAAJ.

[6] Craig Gentry. "A Fully Homomorphic Encryption Scheme". PhD thesis. Stanford University, 2009.

[7] O. Goldreich et al. "Approximating shortest lattice vectors is not harder than approximating closest lattice vectors". In: *Information Processing Letters* 71.2 (July 1999), pp. 55–61. ISSN: 0020-0190. DOI: 10.1016/s0020-0190(99)00083-6.

[8] Oded Goldreich, Shafi Goldwasser, and Shai Halevi. "Public-key cryptosystems from lattice reduction problems". In: *Advances in Cryptology — CRYPTO '97*. Springer Berlin Heidelberg, 1997, pp. 112–131. ISBN: 9783540695288. DOI: 10.1007/bfb0052231.

[9] The LIDIA Group. *LiDIA - a Library for Computational Number Theory*.

[10] Jeffrey Hoffstein, Jill Pipher, and Joseph H. Silverman. *An introduction to mathematical cryptography*. Undergraduate texts in mathematics. Hier auch später erschienene, unveränderte Nachdrucke. New York, NY: Springer, 2010. 523 pp. ISBN: 9781441926746.

[11] Sarah Kamel, Mireille Sarkiss, and Ghaya Rekaya-Ben Othman. "Improving GGH cryptosystem using generalized low density lattices". In: *2016 International Conference on Advanced Communication Systems and Information Security (ACOSIS)*. 2016, pp. 1–6. DOI: 10.1109/ACOSIS.2016.7843914.

[12] Phong Nguyen. "Cryptanalysis of the Goldreich-Goldwasser-Halevi Cryptosystem from Crypto '97". In: *Advances in Cryptology — CRYPTO' 99*. Springer Berlin Heidelberg, 1999, pp. 288–304. ISBN: 9783540484059. DOI: 10.1007/3-540-48405-1_18.

[13] Carl Pomerance. "A Tale of Two Sieves". In: *AMS* 43.12 (Dec. 1996), pp. 1473–1485.

[14] R. L. Rivest, A. Shamir, and L. Adleman. "A method for obtaining digital signatures and public-key cryptosystems". In: *Communications of the ACM* 21.2 (Feb. 1978), pp. 120–126. ISSN: 1557-7317. DOI: 10.1145/359340.359342.

[15] Peter W. Shor. "Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer". In: *SIAM Journal on Computing* 26.5 (Oct. 1997), pp. 1484–1509. ISSN: 1095-7111. DOI: 10.1137/s0097539795293172.

[16] Shuo Sun et al. "Fast Fourier Orthogonalization over NTRU Lattices". In: *Information and Communications Security*. Springer International Publishing, 2022, pp. 109–127. ISBN: 9783031157776. DOI: 10.1007/978-3-031-15777-6_7.

[17] Masayuki Yoshino and Noboru Kunihiro. "Improving GGH cryptosystem for large error vector". In: *2012 International Symposium on Information Theory and its Applications*. 2012, pp. 416–420.